

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА
ФРАНКА**

**Факультет прикладної математики та інформатики
Кафедра програмування**

Лабораторна робота №8

Базові елементи програмування графіки

Виконала:
Студентка групи ПМІ-46
Тригуб Марта

Мета роботи

Вивчити базові принципи програмування комп'ютерної графіки. Побудувати каркас віконного графічного редактора з підтримкою векторного малювання. Реалізувати малювання стандартних геометричних фігур (лінія, прямокутник, еліпс) з інтерактивним попереднім переглядом за рухом миші, а також операції збереження та відновлення малюнку.

Теоретичні відомості

Векторна та растрова графіка

У комп'ютерній графіці існують два принципово різних підходи до збереження зображень: растровий і векторний. Растрове зображення представлене у вигляді двовимірного масиву пікселів (bitmap), а векторне – у вигляді опису геометричних примітивів (координати, колір, товщина). У даній роботі обрано векторний підхід, оскільки він дозволяє зручно зберігати, завантажувати й масштабувати фігури без втрати якості, а також легко скасовувати останню дію.

Метод низхідного програмування (top-down)

Для побудови графічного редактора застосовано метод низхідного проєктування (top-down design). Спочатку визначено повний перелік операцій та меню, а функції наповнювались реальним кодом поступово. Такий підхід дозволяє мати робочий каркас програми з першого кроку і поступово розширювати функціональність.

Бібліотека tkinter

Для реалізації використано мову Python та стандартну бібліотеку tkinter. Малювання фігур здійснюється на компоненті Canvas методами `create_line()`, `create_rectangle()`, `create_oval()`. Інтерактивний попередній перегляд реалізовано через тимчасові об'єкти (preview), які видаляються і перемальовуються при кожній події руху миші.

Опис програмної реалізації

Структура програми

Вся логіка зосереджена в одному класі GraphicsEditor. Конструктор `__init__` ініціалізує стан програми (поточний інструмент, кольори, товщина) та будує інтерфейс через чотири допоміжні методи: `_build_menu()`, `_build_toolbar()`, `_build_canvas()`, `_build_statusbar()`.

Перелік операцій меню

Меню	Операції
Файл	Новий файл, Відкрити..., Зберегти, Зберегти як..., Вийти
Правка	Вирізати, Копіювати, Вставити, Видалити останню фігуру, Вибрати все
Фігури	Лінія, Прямокутник, Еліпс
Властивості	Колір лінії, Колір заливки, Товщина лінії

Малювання фігур за рухом миші

Малювання реалізоване через три обробники подій миші Canvas:

- `on_mouse_down` – фіксує початкові координати фігури.
- `on_mouse_move` – видаляє тимчасову фігуру (`preview_id`) і малює нову від початкової точки до поточної позиції курсора.
- `on_mouse_up` – видаляє `preview`, малює фінальну фігуру та додає її до списку `self.shapes` для збереження.

Ключовим є параметр `tags=("preview",)` при малюванні попереднього перегляду, що дозволяє однозначно ідентифікувати тимчасовий об'єкт і видаляти лише його, не зачіпаючи вже намальовані фігури.

Метод малювання фігур

Уніфікований приватний метод `_draw_shape` обробляє всі три типи фігур:

```
def _draw_shape(self, kind, x1, y1, x2, y2, lc, fc, lw, tags=()):
    kwargs = dict(outline=lc, fill=fc, width=lw, tags=tags)
    if kind == "line":
        return self.canvas.create_line(x1, y1, x2, y2,
                                       fill=lc, width=lw, tags=tags)
    elif kind == "rect":
        return self.canvas.create_rectangle(x1, y1, x2, y2, **kwargs)
    elif kind == "ellipse":
        return self.canvas.create_oval(x1, y1, x2, y2, **kwargs)
```

Збереження та відкриття файлів

Малюнок зберігається у форматі JSON. Кожна фігура представлена словником з полями type, x1, y1, x2, y2, line_color, fill_color, line_width. При відкритті файлу список self.shapes відновлюється і canvas перемальовується методом _redraw_all(). Поле canvas_id не зберігається у файл, оскільки є внутрішнім ідентифікатором об'єкта на полотні.

```
def _write_file(self, path):
    data = {"shapes": [
        {k: v for k, v in s.items() if k != "canvas_id"}
        for s in self.shapes
    ]}
    try:
        with open(path, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
            self.is_modified = False
    except Exception as e:
        messagebox.showerror("Помилка", f"Не вдалося зберегти файл:\n{e}")
```

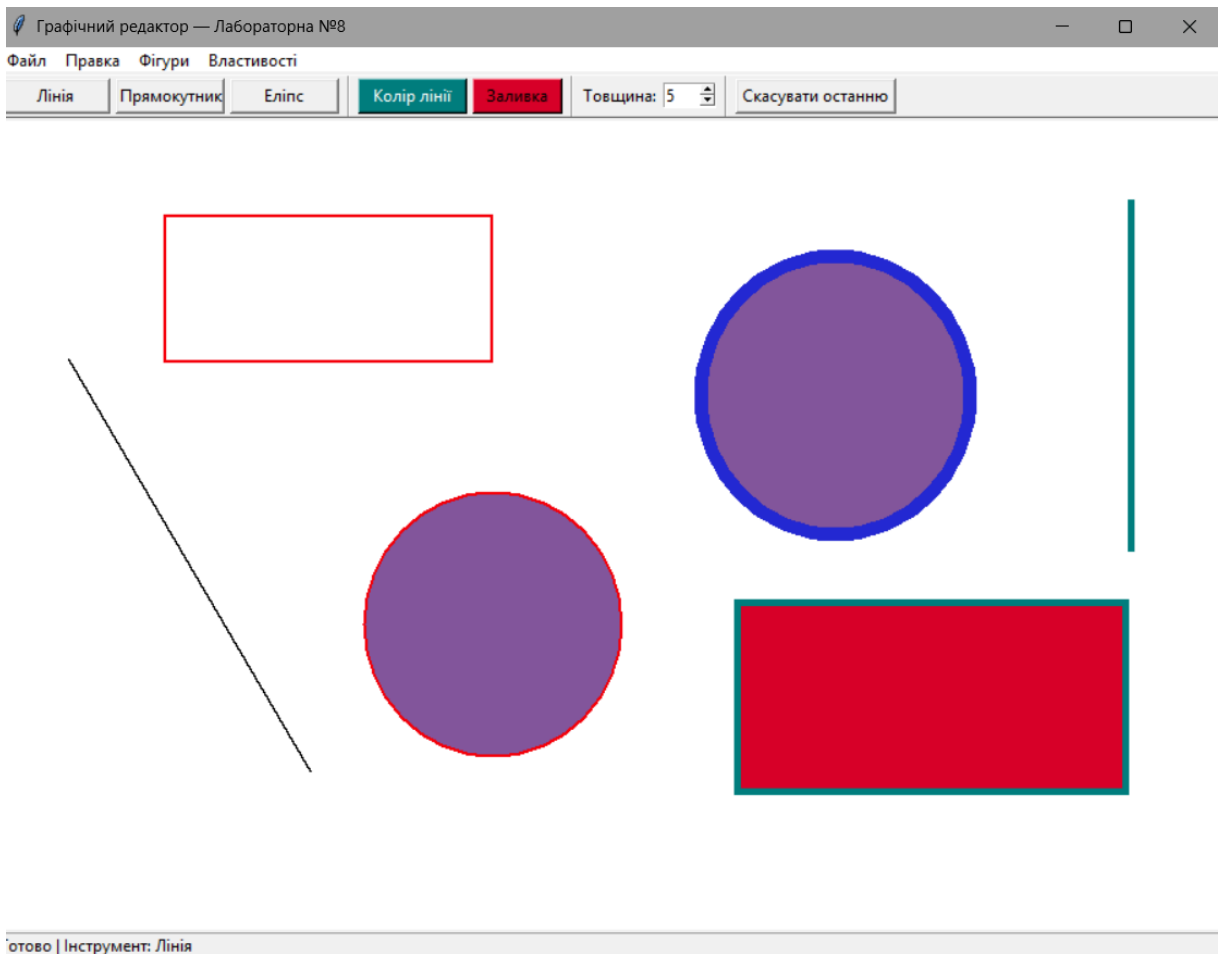
Перевірка збереження при виході

Методи file_new() і app_exit() перед виконанням дії викликають _check_save(). Якщо прапорець is_modified = True, відображається діалог з трьома варіантами: "Так" (зберегти і продовжити), "Ні" (продовжити без збереження), "Скасувати" (повернутися до редактора). Це запобігає випадковій втраті роботи.

Тестування

Для перевірки роботи програми виконано наступні тестові сценарії:

1. Малювання трьох ліній різних кольорів і товщин.
2. Малювання прямокутника з заливкою і без неї.
3. Малювання еліпсу з різними параметрами кольору.
4. Збереження малюнка у файл drawing.json та повторне відкриття.
5. Перевірка скасування останньої фігури кнопкою "Скасувати останню".
6. Перевірка запиту збереження при Новий файл з незбереженими змінами.



Всі сценарії виконано успішно. Попередній перегляд фігур (preview) відображається коректно при русі миші і не залишає артефактів. Після збереження та відновлення малюнка всі фігури відтворюються з точними координатами і параметрами кольору.

Висновки

У ході лабораторної роботи розроблено повнофункціональний каркас графічного редактора мовою Python з використанням бібліотеки tkinter. Реалізовано векторний підхід до збереження зображень, три

типи геометричних фігур з інтерактивним попереднім переглядом, налаштування кольорів і товщини лінії, а також операції збереження/відкриття малюнку у форматі JSON.

Застосований метод низхідного програмування (top-down) виявився ефективним: наявність повного меню і каркасу класу з самого початку дозволила зосередитися на реалізації конкретних функцій, не відволікаючись на архітектурні рішення. Отримані навички є базою для наступних лабораторних робіт з програмування графіки.